

Design Doc: Async Export Pipeline

Background

The reporting service currently generates CSV exports synchronously inside the request handler, which means a large export can hold an HTTP connection open for over a minute and regularly trips the load balancer's 30-second timeout. Support has logged this as the single most common cause of export-related tickets over the last two quarters, and it gets worse as customer data volume grows, not better.

This document proposes moving export generation to an asynchronous job queue, with the client polling a status endpoint until the export is ready to download.

Requirements

REQ-101: An export request must return immediately with a job identifier, not block until the export file is fully generated.

REQ-102: A client must be able to poll job status and retrieve a download URL once the job completes, without any change to authentication already in place.

REQ-103: A failed export job must surface a clear, actionable error to the client, not a silent timeout indistinguishable from still-processing.

Job Queue Design

Decision: jobs are queued in the existing background-worker infrastructure already used for scheduled reports, rather than introducing a new queue technology. Reusing existing infrastructure was chosen over a dedicated export queue because the operational cost of running and monitoring a second queue system was judged not to be justified by export jobs alone.

Supported job states:

- queued
- running
- complete
- failed

Rollout Steps

1. Ship the async endpoint behind a feature flag, default off
2. Enable for internal accounts first and monitor job latency
3. Roll out to all accounts once queue depth stays stable for a week

Job State Comparison

State	Client action	Terminal
queued	keep polling	No
running	keep polling	No
complete	download via URL	Yes
failed	show error, allow retry	Yes

Reference Implementation Sketch

```
def enqueue_export(account_id, filters):  
    job_id = queue.push(ExportJob(account_id, filters))  
    return job_id
```

Failure Modes

WARNING: a client that never polls for status will never learn its export failed. The design must still email a failure notification for jobs older than an hour with no successful poll, so a failure is never silently lost.

NOTE: job records are retained for 24 hours after completion, then purged — this is a deliberate storage-cost trade-off, not an oversight; a client that waits longer than that to download must re-request the export.

TODO: determine the exact retry policy for a transiently-failed job once the async path is live in production — the current single-attempt behavior is a starting point, not a measured decision.

Open Questions

Should very small exports (under some row-count threshold) still be generated synchronously, to avoid the round-trip latency of polling for something that would have completed in under a second anyway? Current lean is no, for consistency of client implementation, but this needs data from the initial internal rollout before being decided for good.